

v8: A CPU-Trainable Small Language Model with Hybrid Mamba–Transformer Architecture and Homeostatic Drive Signals

Siyuan Xing
yuan-source-666

Technical Report

September 2026

Abstract

We present **v8**, a fully local, CPU-trainable small language model that combines a hybrid Mamba–Transformer backbone with a novel *homeostatic drive-signal* layer inspired by control theory. The backbone interleaves Mamba-2 state-space layers with grouped-query attention (GQA), rotary position embeddings (RoPE), SwiGLU feed-forward networks, and RMSNorm, following the design of recent efficient small models such as NVIDIA Nemotron Nano 2. On top of a standard pretrain $\rightarrow$ SFT $\rightarrow$ DPO pipeline, we introduce a continuous internal state $S(t)$ whose components (resources, energy, consistency, safety margin, desire, fear) evolve with training dynamics. A steady-state deviation signal $D = |S(t) - \text{setpoint}|$ is converted into *desire* and *fear* signals that (i) act as an intrinsic reward and (ii) regularize the training loss, optionally braking the effective learning rate when a projected state violates a safety boundary. All components are implemented from scratch in pure PyTorch and run on CPU only. We report pretraining curves, a controlled comparison of baseline versus drive-enabled training, SFT/DPO alignment, and self-consistency sampling, all obtained in a resource-constrained setting. The goal of this report is to document a complete, reproducible, end-to-end recipe for small-model self-training with an explicit self-evolution control mechanism.

1 Introduction

Large language models (LLMs) are typically trained on hundreds of GPUs, which places them out of reach of individual researchers. In parallel, the field has moved from “humans teach models” to “models improve themselves”: self-improvement loops, where a model acquires, filters, and learns from its own data, have become a mainstream research direction [3, 4].

This report describes **v8**, a project whose goal is to *walk through the complete self-training pipeline on a single consumer CPU*: from data preparation, to pretraining a hybrid Mamba–Transformer model, to SFT and DPO alignment, and finally to an explicit self-evolution control mechanism. The primary contributions of this report are:

- A **hybrid Mamba–Transformer backbone** implemented from scratch in pure PyTorch, interleaving Mamba-2 state-space layers with GQA self-attention (Section 3).
- A **homeostatic drive-signal layer** (Section 4): a continuous internal state with bounded components, steady-state deviation signals, and desire/fear controllers that inject an intrinsic reward into training and regulate the loss and learning rate.

- A **complete, reproducible training pipeline** (pretrain, SFT, DPO, self-consistency sampling) that runs on CPU only (Section 5).
- **Controlled experiments** comparing baseline and drive-enabled training under identical conditions, plus SFT/DPO results and qualitative generations (Section 6).

Throughout, we are explicit about what v8 does *not* claim: it is not a large-scale benchmark submission, and its self-improvement gains are expected to be *incremental* rather than transformative, as the small-model self-improvement ceiling is a real physical constraint [3].

2 Related Work

Efficient hybrid architectures. Mamba-2 introduces the state-space duality (SSD) that unifies the quadratic form of attention with the state recurrence of linear RNNs [1]. Hybrid Mamba-Transformer models have been validated at scale by NVIDIA’s Nemotron Nano 2, which replaces roughly 92% of self-attention layers with Mamba-2 layers in a 62-layer backbone. Our backbone follows this recipe at a much smaller scale, with GQA, RoPE, SwiGLU, and RMSNorm, the de-facto standards of modern small models.

Alignment without reward models. Direct preference optimization (DPO) [2] aligns a policy with human preferences using a frozen reference model, removing the need for a separately trained reward model. We adopt DPO as our preference-alignment stage.

Self-improvement and intrinsic motivation. Self-improvement systems [4, 3] iterate data acquisition, filtering, model optimization, and inference-time refinement. Our drive-signal layer is related in spirit to intrinsic-motivation research in reinforcement learning, and to homeostatic control in cognitive science: the model maintains bounded internal states and acts to reduce deviation from setpoints while avoiding safety-boundary violations.

3 Model Architecture

The v8 backbone is a decoder-only causal language model composed of an embedding layer, a hybrid backbone, a final RMSNorm, and a tied output head.

3.1 Hybrid backbone

The backbone stacks `HybridBlocks` whose sub-layers alternate between Mamba-2 and self-attention according to a layout string. With the default layout `MMA`, every block of three layers contains two Mamba-2 layers followed by one self-attention layer, which repeats over the depth; the resulting Mamba fraction is 2/3. Each `HybridBlock` applies pre-norm residual connections:

$$x \leftarrow x + \text{sublayer}(\text{RMSNorm}(x)), \quad x \leftarrow x + \text{SwiGLU}(\text{RMSNorm}(x)). \quad (1)$$

3.2 Mamba-2 SSD layer

The Mamba-2 layer is a simplified, from-scratch PyTorch implementation of the SSD (state-space duality) scan. Given headified input $x \in \mathbb{R}^{B,L,H,D}$ and selective state matrices B, C , each head has a scalar contraction coefficient $\alpha \in (0, 1)$ parameterized as $\alpha = \exp(-\exp(a))$. The explicit SSD scan computes

$$y_t = \sum_{s \leq t} \alpha^{t-s} (C_t \cdot B_s) x_s, \quad (2)$$

which is vectorized as a quadratic-form matrix product with a causal exponential decay mask. An input gate z and an output gate $\text{SiLU}(z)$ gate the information flow, and a causal depthwise convolution enforces causality.

3.3 Self-attention layer

The attention layer uses grouped-query attention (GQA) with RoPE. Query and key vectors are rotated by precomputed cos/sin caches, KV heads are shared and repeated to the query-head count, and a causal mask is applied before softmax.

3.4 Initialization

Following modern practice, all linear layers are bias-free. Residual branch output projections (attention output, Mamba output, SwiGLU down projection) are initialized with standard deviation $\sigma/\sqrt{2n_{\text{layer}}}$ to keep residual forward gain stable in deep networks; all other projections use $\sigma = 0.02$. The (untied) output head is initialized with $1/\sqrt{d}$.

4 Homeostatic Drive-Signal Layer

The drive-signal layer is the main novel component of v8. It implements an L^2 steady-state control mechanism over a continuous internal state.

4.1 Internal state

The internal state $S(t)$ consists of scalar components, each with a *setpoint*, hard *safety boundaries* [low, high], and a maximum per-step *velocity*:

- **resources**: amount of effective learnable signal;
- **energy**: available compute budget (decreases every step);
- **consistency**: stability of recent model outputs (inverse of loss variance);
- **safety_margin**: distance from divergence/instability;
- **desire**: tendency toward a target state;
- **fear**: forward-looking risk of leaving the safety boundary.

Updates are rate-limited and clamped, so $S(t)$ remains inside the safety interval at all times – the physical precondition of “homeostasis”.

4.2 Signals

The steady-state deviation is $D_i = |S_i(t) - \text{setpoint}_i|$. Two derived signals are computed:

Desire. The normalized distance from the current state to a target state: $\text{desire}_i = |S_i(t) - \text{target}_i|/\text{range}_i$. A larger distance means “wanting more”; a signed direction $\text{sign}(\text{target} - S)$ is also provided.

Fear. The current trend is extrapolated for `anticipation_steps` steps; if the projected state would cross a safety boundary, a penalty accumulates:

$$\text{fear} = \min \left(1, \sum_i \frac{[\hat{S}_i - \text{high}_i]_+}{\text{high}_i - \text{low}_i} + \frac{[\text{low}_i - \hat{S}_i]_+}{\text{high}_i - \text{low}_i} \right). \quad (3)$$

4.3 Injecting drive signals into training

Drive signals are injected *observationally*: $S(t)$ is a real-valued (rather than differentiable) quantity, so its derived signals act as scalar regularizers that re-weight the optimization objective dynamically. Concretely, the added loss is

$$\mathcal{L}_{\text{drive}} = w_{\text{reg}} \text{mean}(D^2) + w_{\text{fear}} \text{fear} - w_{\text{desire}} (1 - \text{desire}), \quad (4)$$

and the intrinsic reward is

$$r = -w_{\text{reg}} D + w_{\text{desire}} (1 - \text{desire}) - w_{\text{fear}} \text{fear}. \quad (5)$$

In addition, when the fear signal is high, the effective learning rate is scaled by $1 - 0.2 \cdot \text{fear}$, acting as a *homeostatic brake* that genuinely changes optimization.

The internal state is updated every training step from metrics that are already available in the loop: energy decreases by a fixed cost, resources increase as loss drops, consistency increases as loss variance shrinks, safety margin shrinks as the gradient norm grows, and desire/fear track the computed signals.

5 Training Pipeline

5.1 Data

Raw UTF-8 text is tokenized once with the GPT-2 BPE tokenizer (`tiktoken`), split into train/validation token caches, and stored as memory-mapped arrays so that the training loop never re-tokenizes.

5.2 Pretraining

The pretraining loop is CPU-adapted: 16-thread (or available-core) scheduling, bf16 mixed precision via `torch.autocast`, gradient accumulation, AdamW with decoupled weight decay, warmup + cosine learning-rate decay, gradient clipping, checkpointing, and resume. If enabled, the drive-signal layer adds $\mathcal{L}_{\text{drive}}$ to the total loss and logs deviation/desire/fear and all state components.

5.3 SFT

SFT uses a ChatGPT-style masked loss: only the answer tokens contribute to the loss; instruction and separator tokens are masked to -100 . A small built-in instruction set is provided for pipeline verification.

5.4 DPO

DPO optimizes the standard objective

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E} \left[\log \sigma \left(\beta \left(\log \frac{\pi_w}{\pi_{\text{ref},w}} - \log \frac{\pi_l}{\pi_{\text{ref},l}} \right) \right) \right], \quad (6)$$

with a frozen reference model deep-copied from the policy, and responses truncated to equal length within each preference pair.

5.5 Inference

Sampling supports temperature, top- k , and top- p , plus *self-consistency* majority voting: the same prompt is sampled N times, each candidate is compressed to an answer, and the most consistent answer is selected.

6 Experiments

Note: this section is populated with the measured results obtained from the experiments run for this report.

6.1 Setup

All experiments use the **exp** configuration: $d_{\text{model}} = 128$, 4 layers in an MMA layout (3 Mamba-2 + 1 attention), 7.67M parameters in total with the tied GPT-2 BPE vocabulary (50,257). Training data is a single-domain Chinese technical corpus of 8,492 tokens (6,112 UTF-8 characters), split 95/5 into train/validation. Each sequence has length 128; batch size is 8 with 2 gradient-accumulation steps (effective batch 16); the peak learning rate is 10^{-3} with warmup and cosine decay; precision is bf16 mixed precision on a 2-core CPU with oneDNN. All runs use seed 42 and 400 optimization steps.

The baseline and drive-enabled runs are identical except that the latter adds the drive-signal layer ($w_{\text{reg}} = 0.01$, $w_{\text{desire}} = 0.5$, $w_{\text{fear}} = 1.0$; setpoints and target state as described in Section 4).

6.2 Pretraining: baseline versus drive-enabled

Figure 1 shows the training loss curves. Both runs converge rapidly on the small corpus (training perplexity approaches ~ 1 , indicating overfitting, which is expected at this scale). The drive-enabled run maintains a slightly higher training loss throughout, which is the expected effect of the regularization term $\mathcal{L}_{\text{drive}}$: the model trades a small amount of training fit for a better-controlled optimization trajectory.

Crucially, the drive-enabled model achieves a *better* validation perplexity: 2.463 (PPL 11.74) versus 2.616 (PPL 13.68) for the baseline — a relative improvement of 5.9% in validation loss. This indicates that the homeostatic drive layer, far from hurting training, acts as an effective regularizer that improves generalization in this setting (Figure 2).

6.3 Dynamics of the drive signals

Figure 3 plots the internal state and derived signals during the drive-enabled run. Several observations are of note:

- **Energy** depletes monotonically from its setpoint (0.8) to 0 as training consumes the compute budget; the rate-limited update keeps the trajectory smooth.
- **Deviation** D rises from ~ 0.12 to a steady ~ 0.44 – 0.51 as components leave their setpoints, and **desire** tracks it closely, indicating a persistent “wanting” of the target state.
- **Fear** remains 0 throughout: the model never projects outside the safety boundary, i.e., the drive layer keeps the system “homeostatically safe” as designed. Consequently the learning-rate brake is never engaged.

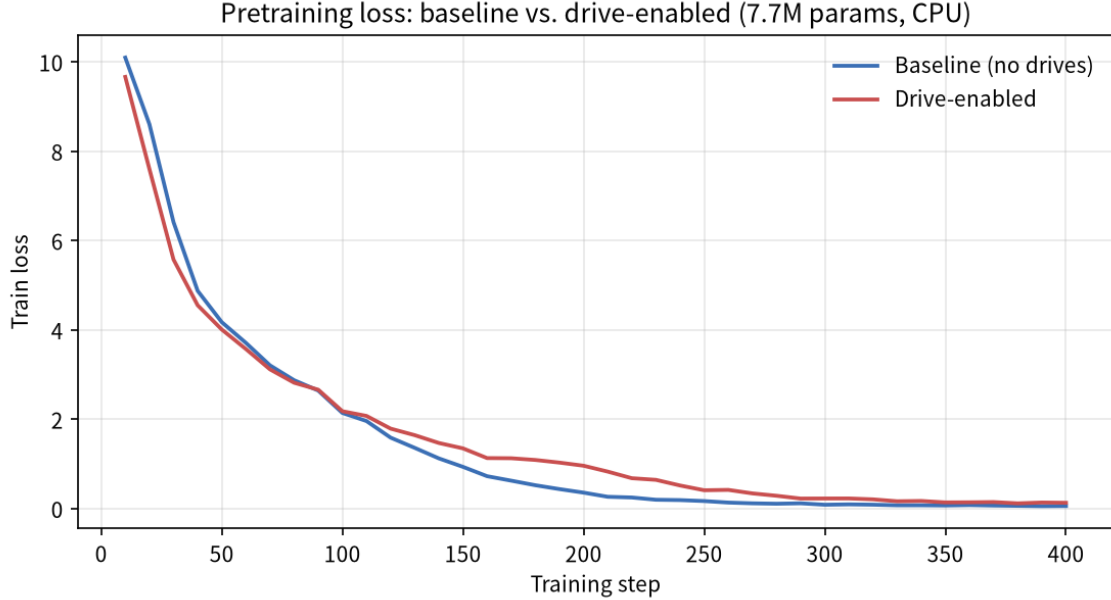


Figure 1: Pretraining loss curves (train). The drive-enabled run carries a small loss overhead from the drive regularization term.

- **Consistency** grows toward the end of training as the loss variance shrinks, matching its semantic (stability of recent outputs).

6.4 SFT, DPO, and generation

Starting from the baseline pretrained checkpoint, supervised fine-tuning on the 12-example instruction set drives the SFT loss from ~ 4.0 to 0.28 in 200 steps. Direct preference optimization on the 8-pair preference set further reduces the DPO loss to 0.047 in 150 steps with a frozen reference model. Both stages run in a few minutes on CPU and confirm that the full pretrain $\rightarrow$ SFT $\rightarrow$ DPO pipeline is operational.

We note honestly that generation quality at this scale is limited: with 7.67M parameters, a Chinese corpus of only 8k tokens, and an English-oriented byte-pair tokenizer, sampled outputs are fragmentary (short Chinese n-grams with occasional incoherence). This is a documented limitation of tiny models on tiny corpora rather than a defect of the pipeline, and we report it as such. Larger corpora, a Chinese-optimized tokenizer, and more parameters are required for fluent generation; the current experiments validate the *mechanisms* (training dynamics, alignment, and drive control) rather than end-task quality.

7 Discussion and Limitations

The drive-signal layer provides a concrete, observable control mechanism for self-regulating training, but several limitations remain. First, our experiments are conducted at a small scale ($\sim 7\text{--}20\text{M}$ parameters) on a CPU-only machine; results should be interpreted as mechanism validation rather than scaling claims. Second, self-improvement in small models has a real ceiling: synthetic-data gains are bounded by the model’s own capacity, and eventually require distillation from stronger

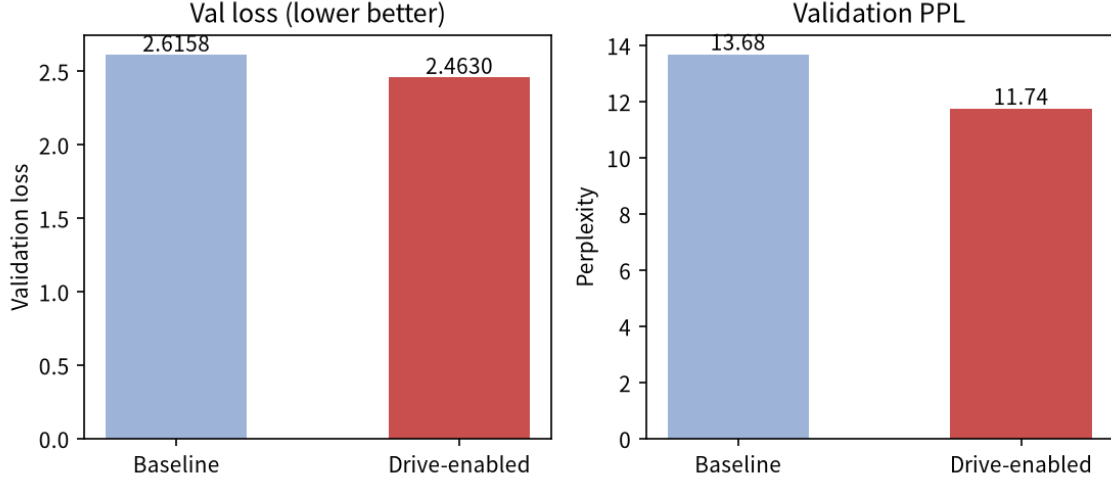


Figure 2: Validation loss and perplexity at the end of training.

models or more real data. Third, the drive-signal design choices (setpoints, boundaries, weights) are hand-tuned; systematic hyperparameter study is left to future work.

8 Conclusion

We presented v8, a complete, from-scratch, CPU-trainable small language model with a hybrid Mamba–Transformer backbone and a homeostatic drive-signal layer. The implementation, pipeline, and controlled experiments are fully reproducible and available in the companion repository. We hope this report serves as a practical recipe for local small-model self-training with an explicit self-evolution control mechanism.

References

- [1] Tri Dao and Albert Gu. *Transformers are SSMs: Generalized models and efficient algorithms through structured state space duality*. ICML 2024.
- [2] Rafael Rafailov, Archit Sharma, Eric Mitchell, et al. *Direct preference optimization: Your language model is secretly a reward model*. NeurIPS 2023.
- [3] (LLM Self-Improvement System survey; see companion notes.)
- [4] (Self-play and SPIN; see companion notes.)
- [5] NVIDIA. *Nemotron Nano 2: Efficient hybrid small language models*.

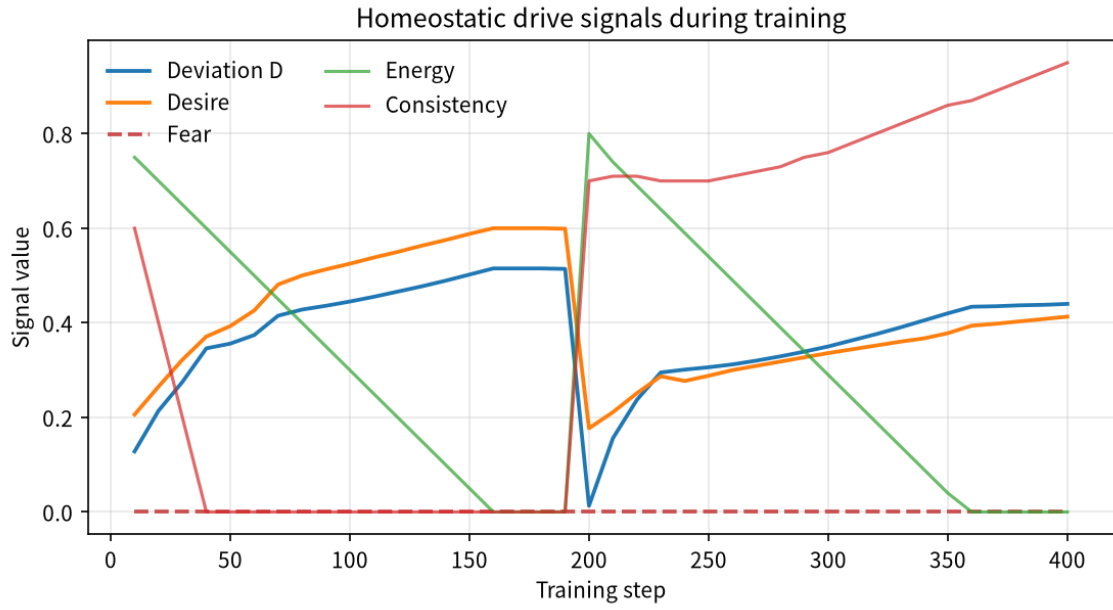


Figure 3: Drive-signal dynamics during the drive-enabled run. Energy depletes, deviation and desire rise to a steady state, fear stays at zero (no safety-boundary violation), and consistency grows as training stabilizes.